

Week 5 — From Readings to Real Objects

Course: Scientific Programming with Python

Timebox: Tutorial 75 min · Exercise 5 ≤ 30 min · Exercises 6–10 = home tasks for ~1 week

Project: The rooftop campus weather-station

Narrative setup

Your team now wants the station’s pieces to feel like real things: sensors that can introduce themselves, readings that carry units, and a station that organizes who does what. This week, you’ll shape tidy “blueprints,” plug them together, and re-use them to keep the project sane as it grows. Keep outputs compact and predictable — your teammates will paste them into checks and small dashboards.

Exercise 1 — A Sensor Introduces Itself

Type: Scientific · Estimated time: short (5–7 min)

A temperature probe (sensor) should present a one-line self-introduction with its label and range in SI units. The team expects exactly: `SENSOR <label> | kind=temp | range=[lowC,highC] °C`.

What you must produce. One `script` that defines a reusable “blueprint” for a probe and then prints the line for a probe labelled CU-PHY-2/T1 with range `[-40, 60]`.

Inputs/Outputs.

- Inputs: label (short text), lowC ∈ `[-100, 0]`, highC ∈ `[0, 100]`.
- Output: exactly one line as above; numbers as integers; include °C.

Reflection. What single source of truth did you establish for the label and range?

Why this matters? Consistent self-descriptions prevent mismatched units later.

Hint: A class `Probe` with `init` and a method `intro()` that returns a string. Sample output of `intro()`: `SENSOR CU-PHY-2/T1 | kind=temp | range=[-40,60] °C`

Exercise 2 — Snapshots that Carry Units

Type: Scientific · Estimated time: short (7–9 min)

A reading should bundle value and unit and be able to present itself like `21.5 °C`. Build such a reading and show it for 21.5 and for 294.65 with unit K.

What you must produce. A reusable way to create a “reading” with a value and a unit and a method to render it. Print two demo lines.

Inputs/Outputs.

- Inputs: value: float in a plausible range; unit: string from {°C, K, %, hPa}.
- Output: one line per reading; temperature with 1 dp (°C) and 2 dp (K); others 0 dp.

Reflection. How does keeping the unit attached to the value reduce mistakes?

Why this matters? Typed outputs (value+unit) avoid confusion in mixed pipelines.

Hint: Reading is a class with value and unit as member data.

Exercise 3 — A Crew Badge for the Station

Type: General · Estimated time: short (7–9 min)

The station should have a name and keep a short roster of who can operate it today. It must answer with a compact line: STATION <name> | roster=N.

What you must produce. A reusable station “blueprint” that holds a name of the station and a small list of operators – first names – (strings) and can report its roster size in the exact format.

Inputs/Outputs.

- Inputs: name: short text; operators: 1–10 names (strings).
- Output: one line exactly as shown with correct N.

Reflection. Why keep the roster with the station instead of passing it around?

Why this matters? Grouping related data with the thing that owns it improves clarity.

Hint: a class Station which holds name and a list of operators of the station. A method “badge” returns a sample output like this:

STATION CU-PHY-2 | roster=3

for an instance of Station initialized as Station("CU-PHY-2", ["Alex", "Ben", "Chen"])

Exercise 4 — Specializations: Two Kinds of Probes

Type: Scientific · Estimated time: short (9–12 min)

The station uses two probe (sensor) types that **share** identity and range (low, high) but differ in how they present themselves: a temperature probe (kind=temp) and a humidity probe (kind=hum). Both must reuse the shared parts.

What you must produce. Two specialized probe variants that inherit the common identity/range behaviour and only change the kind in the printed line. Print one line for each probe (sensor) type.

Inputs/Outputs.

- Inputs: labels as text; valid ranges: temp [-40,60] °C; hum [0,100] %.
- Output: two lines, e.g.,

```
SENSOR CU-PHY-2/T1 | kind=temp | range=[-40,60] °C
```

and

```
SENSOR CU-PHY-2/H1 | kind=hum | range=[0,100] %
```

Reflection. What stayed the same across probe types, and what changed?

Why this matters? Reusing shared behaviour reduces duplication and errors.

Hint: define three classes, BaseProbe, TempProbe, HumProbe. Each class has an “intro” method that contributes partly to the corresponding output.

Exercise 5 — “One Minute, One Report”

Type: Scientific · Estimated time: ≤ 30 min

For one minute, the station aggregates: a station name, one temperature probe and one humidity probe (with their intros), and two readings (e.g., 21.4 °C, 55 %). Produce one summary line:

```
<station>|<temp-probe-intro>|<hum-probe-  
intro>|values=[tempC,hum%]|ok_flags=[temp_ok,hum_ok]  
Temp is ok if in [18,26] °C, humidity ok if in [20,80] %.
```

What you must produce. A small, reusable setup that creates the station + probes, formats both probe intros via their own behaviour, evaluates the two ok-flags using the thresholds, and prints exactly one line as above.

Inputs/Outputs.

- Inputs at runtime: station name (text), tempC ∈ [-40,60], hum% ∈ [0,100].
- Output: exactly one line; tempC with 1 dp; hum% as integer; booleans are True/False.

Reflection. Where did you place the “knowledge of thresholds” so it’s easy to adjust later?

Why this matters? Integrated reports are snapshots teams can trust.

Hint: a sample minute report will look like this:

```
CUPHY|SENSOR CU-PHY-2/T1 | kind=temp | range=[-40,60] °C|SENSOR CU-PHY-2/H1 |  
kind=hum | range=[0,100] %|values=[21.4,55]|ok_flags=[True,True]
```

Exercise 6 — A Day's Cast List

Type: General · Estimated time: home task (30–45 min)

The station logs which person touched which probe. Build a tidy “interaction” record (who, which device label, short action note ≤ 20 chars) and a host that can store multiple interactions and then present the last 3.

What you must produce. Reusable blueprints for an interaction and a host that can accept many and later print the last three in order, one per line.

Inputs/Outputs.

- Inputs: 5–20 interactions (strings for who/label/note).
- Output: up to three most recent lines as-is (if fewer than 3 exist, print all).

Reflection. Why is it useful to centralize how interactions are recorded and shown?

Why this matters? Predictable logs help audits and debugging.

Hint: a sample output may look like the following:

Ben -> CU-PHY-2/H1 :: check

Chen -> CU-PHY-2/T1 :: calibrate

Dana -> CU-PHY-2/H1 :: wipe

Exercise 7 — Calibrated Thermometer with a Safety Gate

Type: Scientific · Estimated time: home task (40–60 min)

Extend the temperature probe so it can produce a reading in either °C or K, based on a short option at the moment of presentation. If given a value outside its declared range, it should politely refuse and signal a problem to the caller, which must continue gracefully.

What you must produce. An extended temperature probe that can render a value in the requested unit and a small driver that calls it on 5 values (some out of range) and never crashes; print a friendly note when a value is refused.

Inputs/Outputs.

- Inputs: 5 values in °C (some outside $[-40, 60]$); target unit $\in \{^{\circ}\text{C}, \text{K}\}$ per call.
- Output: either a formatted reading or a friendly note per value; run continues.

Reflection. Where should refusal happen so the rest of the run is unaffected?

Why this matters? Clear fault paths and unit handling are everyday realities.

Hint: remember to use exception handling. A sample output for **tests = [21.0, -50.0, 0.0, 59.9, 75.0]** may look like the following:

```
21.0 °C
refused: value -50.0 outside [-40,60] °C
0.0 °C
59.9 °C
refused: value 75.0 outside [-40,60] °C
```

Exercise 8 — Two Kinds of Crew, One Station (home task)

Type: General · Estimated time: home task (40–60 min)

The roster has two roles that share name/id but differ in stipend computation: hourly workers (hours × rate) and fixed-stipend workers (weekly amount). The station should ask each member for the amount in the same way and total it.

What you must produce. Two specialized crew types that reuse shared identity and each answers with its own amount; a coordinator that asks each and prints the total as `total_stipend=....`

Inputs/Outputs.

- Inputs: 3–6 crew entries with plausible numbers.
- Output: exactly one line with the total; 2 dp.

Reflection. How does asking each member “the same question” simplify the coordinator?

Why this matters? Uniform interfaces enable simple aggregations.

Exercise 9 — Station Health: Simple Checks

Type: Scientific · Estimated time: home task (40–60 min)

Define a small “health check” host that runs three independent checks: (a) temp in [18,26] °C, (b) humidity in [20,80] %, (c) a note contains no commas. It should collect the pass/fail booleans and print `health=[t_ok,h_ok,note_ok]` without stopping on the first failure.

What you must produce. A reusable checker that gathers the three booleans and prints them in order; demonstrate with three different triplets of inputs.

Inputs/Outputs.

- Inputs: tempC, hum%, note (≤ 60 chars, may include spaces).
- Output: exactly one line `health=[True,False,True]` style.

Reflection. Why is it better to report all failures together than to stop at the first?

Why this matters? Batch checks save time during fieldwork.

Exercise 10 — A Tiny Library of Things

Type: General · Estimated time: home task (45–60 min)

Move the blueprints from Exercises 1–4 into a single small “library” file and write a separate caller that assembles a station with two probes, prints their intros, and then prints one values-summary line as in Exercise 5.

What you must produce. Two files: one library with the blueprints; one caller that imports them and produces the three lines.

Inputs/Outputs.

- Inputs: labels/ranges as in earlier tasks; one temp and one humidity value.
- Output: three lines: station intro, temp-probe intro, hum-probe intro; then the integrated summary.

Reflection. What became simpler in the caller once the library existed?

Why this matters? Modular reuse prevents drift across notebooks and scripts.